

- 1) Creare una classe **Point2D** per rappresentare punti geometrici in 2D. **Point2D** incapsula le coordinate x e y. Oltre ai soliti metodi per l'incapsulamento, prevedere un metodo *toString()* che descriva il punto come segue: "(<x>, <y>)".
Create quindi una classe **Point3D** che estende la classe **Point2D** per incapsulare le coordinate x, y, z. Prevedere un metodo *toString()* che descriva un punto 3D come: "(<x>, <y>, <z>)".
Scrivete infine un programma di esempio in cui create alcuni oggetti delle due classi e li visualizzate utilizzando i metodi *toString()*.
- 2) Costruire una gerarchia di classi che consenta di rappresentare le seguenti entità: **Shape**, **Circle**, **Rectangle**, **Square**.
Shape è caratterizzato da una stringa rappresentante un colore (*color*) e da una variabile *boolean* che indica se è una figura piena o no (*filled*).
Circle è caratterizzato da un raggio (*radius*). **Rectangle** è caratterizzato da *width* e *length*. **Square** è un caso particolare di **Rectangle** in cui base e altezza sono uguali.
In ogni classe, prevedere opportuni metodi per l'incapsulamento dei dati e metodi per determinare area e perimetro (nel caso di un **Circle** il perimetro sarà la circonferenza). Prevedere anche i metodi *toString()*.

ESERCIZI DEL LIBRO DI TESTO

Progetto6.2 JTombola (Versione1.6)

Partendo dalla soluzione del progetto 5.2, incapsulare la classe Bussolotto nel modo ritenuto più corretto (decidere se inserire i metodi setter e getter) e inserirla nel package `com.jtombola`. Nello stesso package creare il tipo Tabellone tenendo presente che è formato da 6 schede da 15 numeri (non consecutivi) come mostrato nella figura seguente.

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90

Nella classe Tabellone aggiungere anche il metodo *selezionaNumero* che seleziona il numero passato come argomento (scegliere il modo più opportuno). La classe Tabellone deve essere una classe che implementa il pattern Singleton. Creare anche la classe Croupier che definisce un metodo *estraiNumero* che prima invoca il metodo *dammiNumero* della classe Bussolotto, e poi lo passa come argomento al metodo *selezionaNumero* della classe Tabellone. Creare una classe *TestCroupier* che testa il metodo *estraiNumero*. Per verificare che tutto funzioni correttamente, inserire delle istruzioni di stampa all'interno solo della classe Croupier, come se spiegasse tutto quello che accade. Esempio di output:

Estraggo un numero dal bussolotto...

Estratto il numero 10.

Seleziono il numero 10 sul tabellone...

Numero 10 selezionato sul tabellone.

Esercizio 7.18 Dati Jmmobiliare

Creare una applicazione chiamata "Jmmobiliare" che permetta di ricercare immobili. In particolare, si potranno vendere appartamenti, garage e palazzi. Creare una gerarchia sigillata di tipi che rappresentano i concetti prima espressi, all'interno di un package chiamato `com.jmmobiliare.dati`. Tenere conto che un palazzo è formato da una collezione di appartamenti. Inserire nelle varie classi i giusti attributi, metodi e costruttori, utilizzando i concetti imparati sino ad adesso (incapsulamento, super, this, *toString*, i modificatori corretti ecc.) Infine, creare un diagramma delle classi che rappresenta questi tipi con tutti i dettagli come documentazione.

Esercizio 7.4

Descrivere tutte le relazioni di ereditarietà tra le seguenti classi: Professore, Studente, Persona, Cattedra, Corso, Aula e Lezione

Esercizio 7.7

Creare un class diagram che mostri le relazioni gerarchiche tra le seguenti classi: Animale, Aquila, Gatto, Mammifero, Pappagallo, Uccello, Volpe

Esercizio 7.8

Usare la generalizzazione per creare una gerarchia che metta in relazione di estensione le classi: Berlina, BiciclettaDaCorsa, MountainBike e SUV. Rappresentarle anche con un diagramma delle classi, che mostri anche gli attributi e i metodi delle classi riportate

Esercizio 7.9 Implementazione dell'ereditarietà

Partendo dalla soluzione dell'esercizio precedente, creare e compilare un progetto Java che implementa il diagramma delle classi